

6a) WAP to implement single link list with following operations: Sort the linked list, Reverse the linked list, Concatenation of two linked lists.

PSEUDO CODE:

void bubbleSort():

struct node *i, *j, *tempdata

for (i = head; i->next != NULL; i = i->next):

for (j = i->next; j != NULL; j = j->next):

if (i->data > j->data):

tempdata = i->data

i->data = j->data

j->data = tempdata

END if

END INNER FOR

END OUTER FOR.

END FUNCTION.

void reverse():

struct node *prev = NULL, *cur = head, *next = NULL

while (cur != NULL) {

next = cur->next

cur->next = prev

prev = cur

cur = next

return prev;

void concatenation():

struct node *head1, *head2, *temp

if (head2 == NULL)

return head1.

temp = head1.


```
while (temp->next != NULL) {
    temp = temp->next;
}
temp->next = head2;
return head1;
```

3.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head1 = NULL;
struct node *head2 = NULL;

void createlist(struct node **head, int n) {
    struct node *newnode, *temp = NULL;
    int data, i;
    if (n <= 0) {
        printf("n should be greater than 0.");
        return;
    }
    for (i = 1; i <= n; i++) {
        newnode = (struct node *) malloc (sizeof (struct node));
        if (newnode == NULL) {
            printf("Memory allocation failed.");
            return;
        }
        printf("Enter data: ");
        scanf("%d", &data);
        newnode->data = data;
        newnode->next = NULL;
```



```

if (*head == NULL) {
    *head = newnode;
}
else {
    temp->next = newnode;
}
temp = newnode;
}
print ("linked list is created.");
}

```

```

void bubbleSort (struct node *head) {
    struct node *i, *j, *tempdata;
    if (head == NULL) {
        return;
    }
    for (i = head; i->next != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                tempdata = i->data;
                i->data = j->data;
                j->data = tempdata;
            }
        }
    }
}

```

```

struct node *reverse (struct node *head) {
    struct node *prev = NULL, *curr = head, *next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
    }
}

```



```

        cur = next;
    }
    return prev;
}

struct node* concatenate(struct node* head1, struct node* head2) {
    struct node* temp;
    if (head2 == NULL) {
        return head1;
    }
    if (head1 == NULL) {
        return head2;
    }
    temp = head1;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = head2;
    return head1;
}

```

```

void display(struct node* head) {
    struct node* temp = head;
    if (head == NULL) {
        printf("List is empty.");
    }
    else {
        printf("Linked list: \n");
        while (temp != NULL) {
            printf("%d -> ", temp->data);
            temp = temp->next;
        }
    }
}

```



```

int main() {
    int ch, n, listChoice;
    do {
        printf("\n 1) Create linked list 2) Sort linked list  

        3) Reverse linked list 4) Concatenation  

        5) Display list");
        printf("\n Enter your choice:");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("list choice (1 or 2):");
                scanf("%d", &listChoice);
                printf("Enter no. of nodes:");
                scanf("%d", &n);
                if (listChoice == 1) {
                    createlist(&head1, n);
                }
                else if (listChoice == 2) {
                    createList(&head2, n);
                }
                else {
                    printf("invalid list number");
                }
                break;
            case 2:
                printf("Sort list (1 or 2):");
                scanf("%d", &listChoice);
                if (listChoice == 1) {
                    bubbleSort(&head1);
                }
                else if (listChoice == 2) {
                    bubbleSort(&head2);
                }
        }
    } while (ch != 0);
}

```



```

    }
    else {
        printf("Invalid list");
        break;
    }
    case 3:
        printf("Reverse list (1 or 2):");
        scanf("%d", &listchoice);
        if (listchoice == 1) {
            head1 = reverse(&head1);
        }
        else if (listchoice == 2) {
            head2 = reverse(&head2);
        }
        else {
            printf("Invalid list");
            break;
        }
    case 4:
        head1 = concatenate(head1, head2);
        break;
    case 5:
        printf("In list 1:");
        display(head1);
        printf("In list 2:");
        display(head2);
        break;
    default:
        printf("Invalid input.");
    }
    while (ch != -1);
}

```


Output:

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 1

List choice (1 or 2): 1

Enter no. of nodes: 3

Enter data: 56

Enter data: 10

Enter data: 32

Linked list is created

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 1

List choice (1 or 2): 2

Enter no. of nodes: 5

Enter data: 34

Enter data: 6

Enter data: 89

Enter data: 57

Enter data: 12

Linked list is created

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 2

Sort list (1 or 2): 1

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 5

List 1: Linked list:

10 → 32 → 56 →

List 2 - linked list:

34 → 6 → 89 → 57 → 12 →

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 3

Reverse list (1 or 2): 1

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 5

List 1: linked list:

56 → 32 → 10 →

List 2: linked list:

34 → 6 → 89 → 57 → 12 →

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 4

1) Create linked list 2) Sort linked list 3) Reverse linked list 4) Concatenation 5) Display list

Enter your choice: 5

List 1: linked list:

56 → 32 → 10 → 34 → 6 → 89 → 57 → 12 →

List 2: linked list:

34 → 6 → 89 → 57 → 12 →